

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Technical Presentation

Weightage: 20%

QUESTIONS:

Total Marks: 20

1. Servlet Architecture & Workflow

Question:

Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

Focus Areas:

- Client–Server interaction
- Role of Web Container
- Request and Response objects

2. Servlet Life Cycle Analysis

Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Focus Areas:

- Life cycle phases
- Execution flow
- Performance implications

3. GET vs POST Method Evaluation

Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
- Security aspects
- URL limitations

4. Session Tracking Techniques

Question:

Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting).

Evaluate their advantages and limitations.

Focus Areas:

- Session persistence
- Security concerns
- Use-case comparison

5. Cookies vs Sessions – Critical Analysis

Question:

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases.

Focus Areas:

- Storage location
- Security risks
- Performance considerations

Code of Conduct:

I VARSHA S 192462003 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals

Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

AIM

To study and present the architecture, request–response cycle, life cycle, HTTP GET and POST methods, and session tracking mechanisms of Java Servlets, and to analyze their role in developing dynamic web applications.

OUTPUT

1. Servlet Architecture & Workflow

Introduction

Java Servlet is a server-side Java technology used to create dynamic web applications. A servlet receives requests from clients, processes the request on the server and sends a response back to the client.

Servlet Architecture

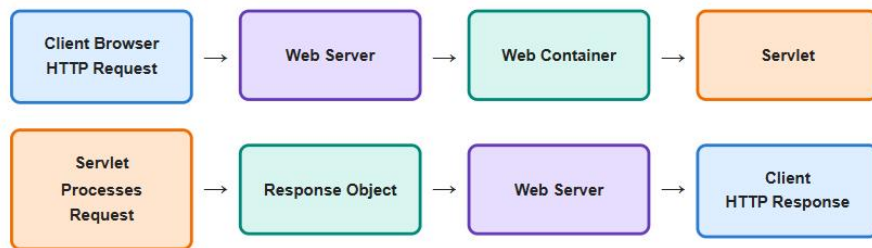
The main components of servlet architecture are the client, web server, web container and servlet. The web container manages servlet execution, including servlet creation, request handling and destruction.



Request–Response Cycle

The request-response cycle begins when a client sends an HTTP request. The web server receives the request and forwards it to the web container. The container identifies the appropriate servlet and creates the `HttpServletRequest` and `HttpServletResponse` objects.

The servlet processes the request and generates the required response. The web container sends this response through the web server back to the client browser.



Role of Request and Response Objects:

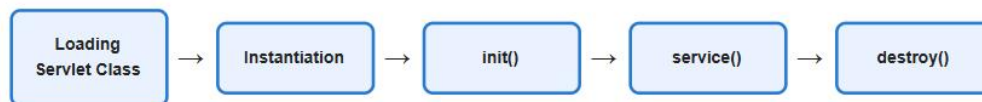
HttpServletRequest: Contains information sent by the client such as form parameters, headers, cookies and request method.

HttpServletResponse: Used by the servlet to send output, status codes, headers and other response information to the client.

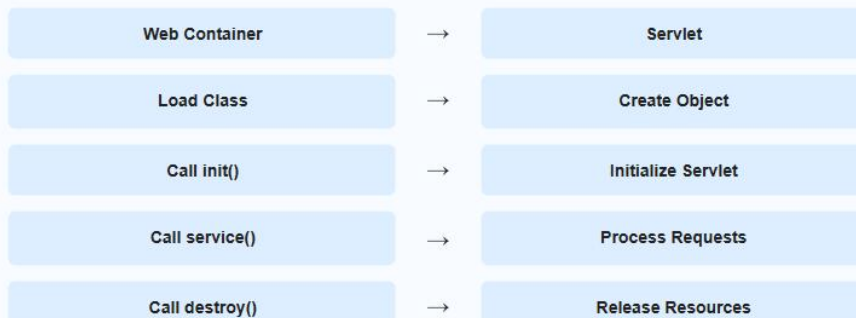
2. Servlet Life Cycle Analysis

Servlet Life Cycle

The servlet life cycle describes the stages through which a servlet passes from creation to destruction. The web container controls the complete life cycle.



Servlet Life Cycle Sequence



Method	Purpose	Execution	Performance Impact
init()	Initializes the servlet and performs one-time setup.	Called once during servlet initialization.	Expensive initialization should be performed carefully because it can increase startup time.
service()	Processes incoming client requests.	Called for each request and may execute many times.	Efficient request processing is important because this method handles application traffic.
destroy()	Releases resources and performs cleanup.	Called once before the servlet is removed.	Proper cleanup prevents resource leaks.

Performance Insight: The servlet object is normally reused by the web container for multiple requests. Therefore, creating expensive objects repeatedly inside `service()` should be avoided when possible. Initialization can be performed once in `init()`, while cleanup should be performed in `destroy()`.

3. GET vs POST Method Evaluation

GET Method

GET is primarily used to request or retrieve data from a server. The form data is appended to the URL as query parameters. It is suitable for searches, filters and retrieving resources where the operation does not change server-side state.

POST Method

POST sends form data in the HTTP request body. It is commonly used when submitting data to the server, creating resources or performing operations that change server-side state.

Feature	GET	POST
Data Location	Data is sent through the URL query string.	Data is sent in the HTTP request body.
Visibility	Parameters can appear in the browser URL and history.	Parameters are not normally displayed in the URL.
URL Length	Limited by browser/server URL length limits.	Not constrained by URL length in the same way.
Bookmarking	Easy to bookmark because parameters are in the URL.	Normally not suitable for bookmarking submitted data.
Typical Usage	Search, filtering, viewing resources.	Registration, login submission, uploading or creating data.
Security	Sensitive data should not be placed in the URL.	Better for sensitive form data, but HTTPS is still required.
Example	Search products or view a page.	Submit registration information.

4. Session Tracking Techniques

HTTP is stateless, meaning that each request is treated independently. Session tracking techniques allow a web application to associate multiple requests with the same user or session.

1. HttpSession

HttpSession is a server-side mechanism that allows the application to store session information associated with a particular user.

Advantages

- Easy to use with Java Servlets
- Data is maintained on the server
- Suitable for login and shopping cart information

Limitations

- Uses server memory
- Requires session management
- Large numbers of sessions can increase server resource usage

2. Cookies

Cookies are small pieces of information stored by the browser and sent with subsequent requests to the associated website.

Advantages

- Simple implementation
- Can persist across requests
- Useful for preferences and identifiers

Limitations

- Stored on the client
- Can be deleted or blocked
- Sensitive data should not be stored directly

3. URL Rewriting

URL rewriting adds a session identifier or required information to the URL when cookies are unavailable.

Advantages

- Works when cookies are disabled
- Simple fallback technique

Limitations

- Session IDs may appear in URLs
- URLs become longer
- Sharing URLs may expose session information

Session Tracking Comparison

Technique	Storage	Advantage	Limitation	Use Case
HttpSession	Server	Convenient server-side session management	Consumes server resources	Login sessions and shopping carts
Cookies	Client Browser	Simple and persistent	Can be disabled or manipulated	User preferences and session identifiers
URL Rewriting	URL	Works without cookies	Session ID may be exposed in URL	Fallback when cookies are unavailable

5. Cookies vs Sessions – Critical Analysis

Cookies and sessions are commonly used to maintain state in web applications. They differ mainly in where information is stored and how the server identifies the user.



Feature	Cookies	Sessions
Storage Location	Client/browser	Server-side
Data Capacity	Small amount of data	Can hold more server-side session state depending on application design
Security	Client-side storage introduces risks if sensitive data is stored improperly.	Sensitive state can remain on the server, but session IDs must be protected.
Performance	Sent with relevant requests, adding network overhead.	Requires server-side session storage and lookup.
Persistence	Can be configured to persist after the browser is closed.	Usually associated with a server-side session lifecycle.
Common Use	Preferences, language selection, identifiers.	Login state, shopping carts, temporary user information.

Real-Time Use Cases

Online Shopping

A session can maintain the user's shopping cart while they move between different pages of an online store.

Online Banking

A secure session can associate authenticated requests with the logged-in user. HTTPS and secure session management are essential.

Website Preferences

Cookies can remember non-sensitive preferences such as language, theme or selected settings.

RESULT

The architecture and complete life cycle of Java Servlets were successfully studied and presented. The GET and POST methods, along with Cookies, HttpSession, and URL rewriting techniques, were compared based on their usage, security, storage, and performance. Thus, the concepts of server-side request processing and session management in dynamic web applications were understood successfully.